



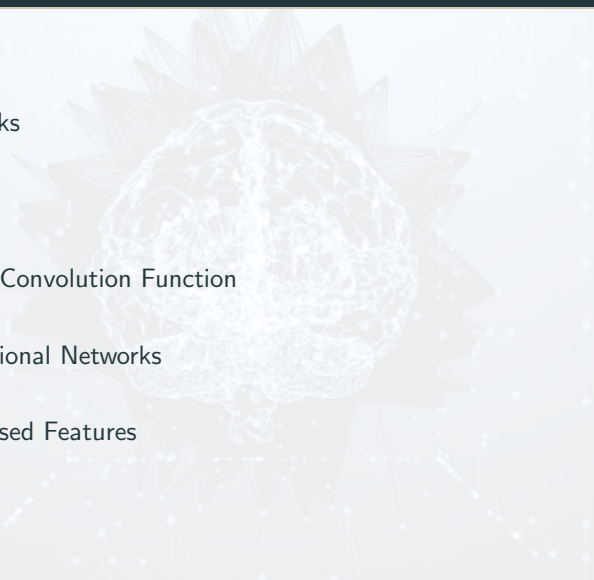
Deep Learning

Convolutional Networks

Lecturer: Duc Dung Nguyen, PhD.

Contact: nddung@hcmut.edu.vn

Faculty of Computer Science and Engineering
Hochiminh city University of Technology

- 
- A faint, stylized background graphic of a human brain with neural connections, rendered in a light gray color, centered behind the list of contents.
1. Convolutional Networks
 2. Pooling
 3. Variants of the Basic Convolution Function
 4. Problems of Convolutional Networks
 5. Random or Unsupervised Features
 6. Case Study

Convolutional Networks



- **Convolutional networks** (**Convolutional Neural Networks**, or **CNNs**: a specialized kind of neural network for processing data that has a known, **grid-like topology**)
- **Convolution**: a specialized kind of linear operation
- **CNNs**: are simply NNs that use **convolution** in place of general matrix multiplication in at least one of their layers

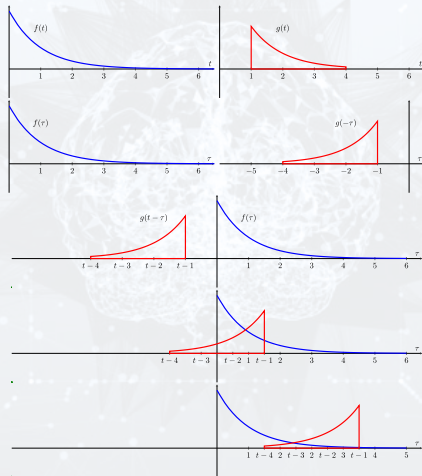
- Signal processing

$$\begin{aligned} s(t) &= \int x(\tau)w(t - \tau)d\tau \\ &= \int x(t - \tau)w(\tau)d\tau \end{aligned} \tag{1}$$

- The convolution operation is typically denoted with an asterisk:

$$s(t) = (x * w)(t) \tag{2}$$

The Convolution Operation



- The continuous signal is unrealistic
- Discrete convolution

$$s(t) = (x * w)(t) = \sum_{\tau=-\infty}^{\infty} x(\tau)w(t - \tau) \quad (3)$$

- In ML applications
 - The input is usually a multidimensional array of data
 - The kernel is usually a multidimensional array of parameters: are adapted by the learning algorithm

The Convolution Operation



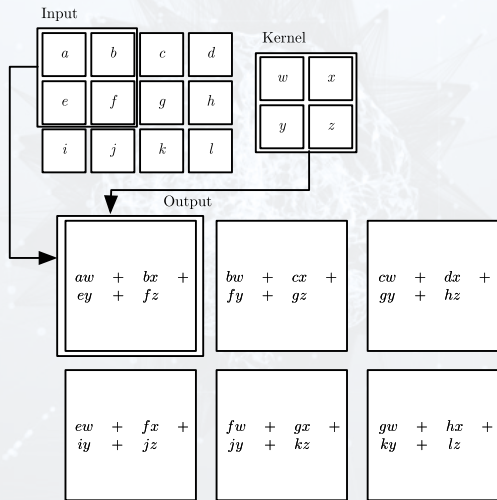
- We often use convolutions over more than one axis at a time
- If we have a 2D image I , we also need a 2D kernel K

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n). \quad (4)$$

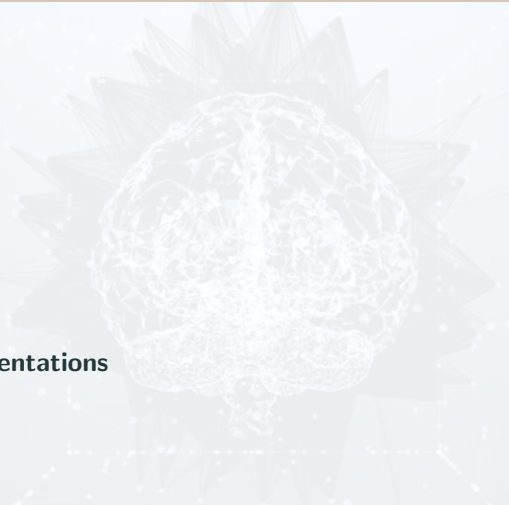
- Convolution is commutative:

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n). \quad (5)$$

The Convolution Operation



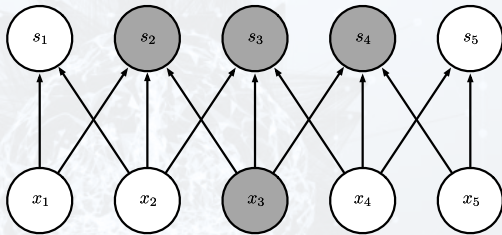
- Many ML frameworks implemented convolution as cross-correlation
 - **Misleading**
- ML algorithm: learn the appropriate values of the kernel in the appropriate place
- An algorithm based on convolution will learn a kernel that is flipped relative to the kernel learned by an algorithm without flipping.

- 
- A large, faint, stylized graphic of a human brain is centered in the background. The brain is composed of a complex network of white lines representing neural connections, set against a light gray background with a subtle grid of dots and lines.
- **Sparse interactions**
 - **Parameter sharing**
 - **Equivariant representations**

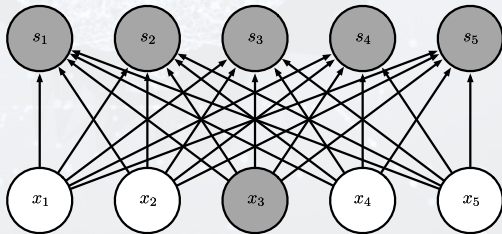
Sparse interactions

- Traditional networks: every output unit interacts with every input unit
- Large kernels $\rightarrow$ Small kernels
 - Memory efficiency
 - Computation requires fewer operations
 - Units in deeper layers may indirectly interact with larger portion of the input

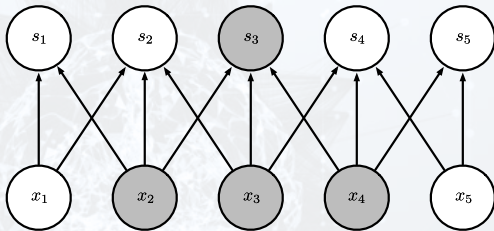
Sparse
connections
due to small
convolution
kernel



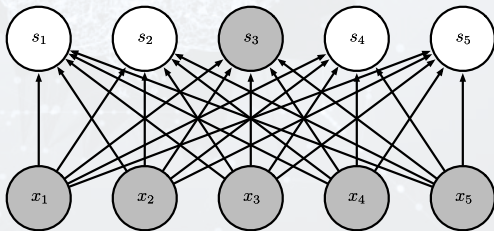
Dense
connections

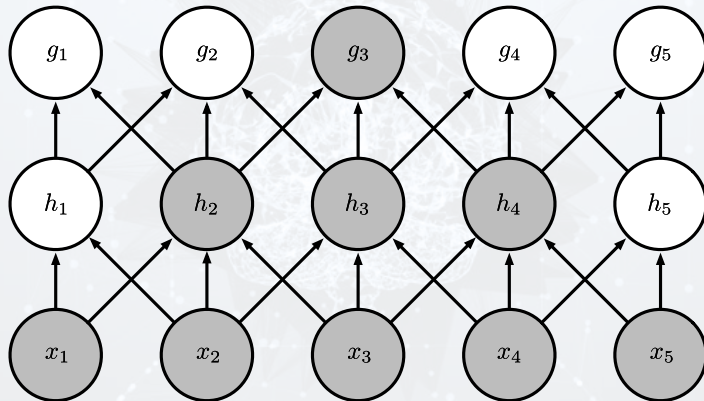


Sparse
connections
due to small
convolution
kernel



Dense
connections





Parameter Sharing

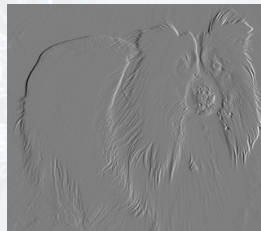
- Using the same parameter for more than one function
- Learn a set of parameters for every location
- Does not reduce the runtime of forward propagation but the storage requirement



Input

1	-1
---	----

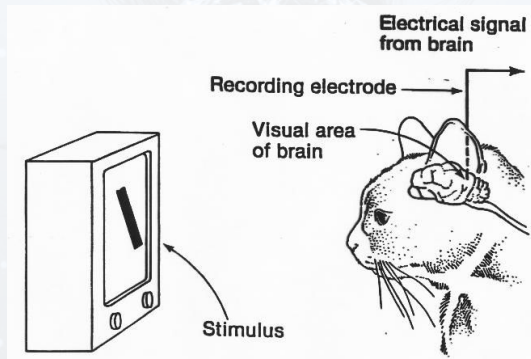
Kernel

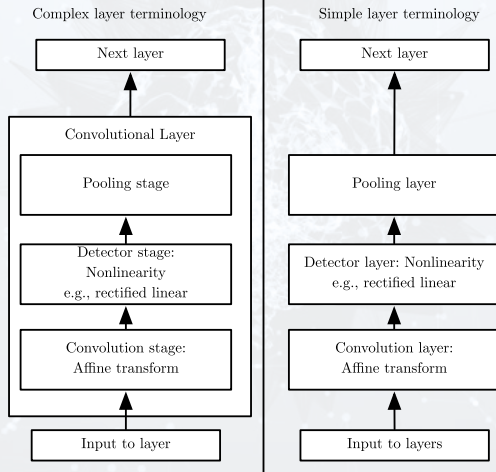


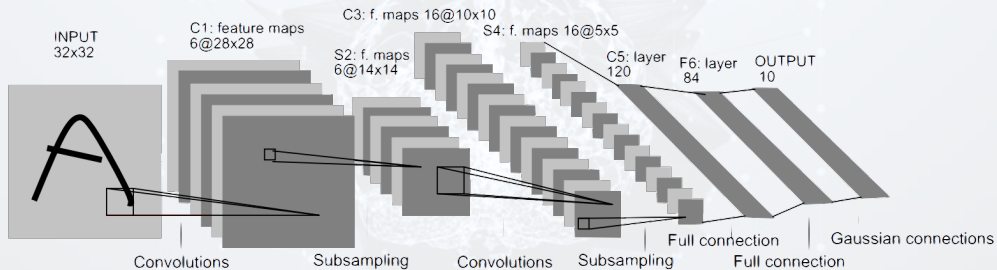
Output

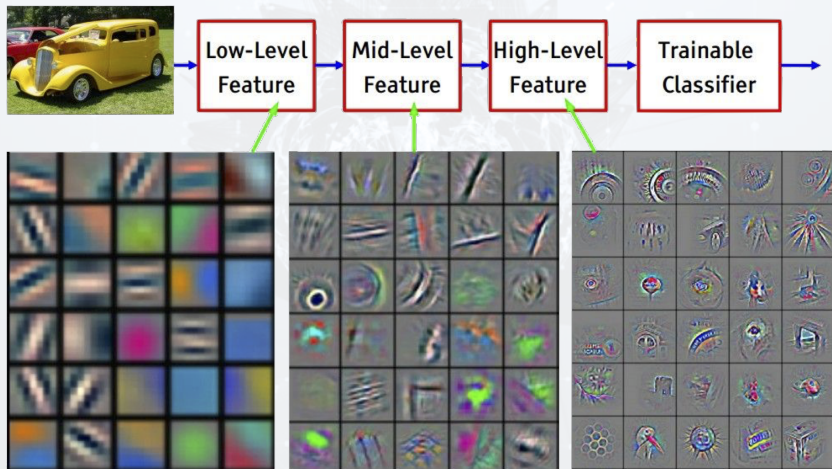
Equivariance

- A function is equivariant means that if the input change, the output changes in the same way
- Function f is equivariant to function g if $f(g(x)) = g(f(x))$
- Convolution is not naturally equivariant to transformations (except translation)





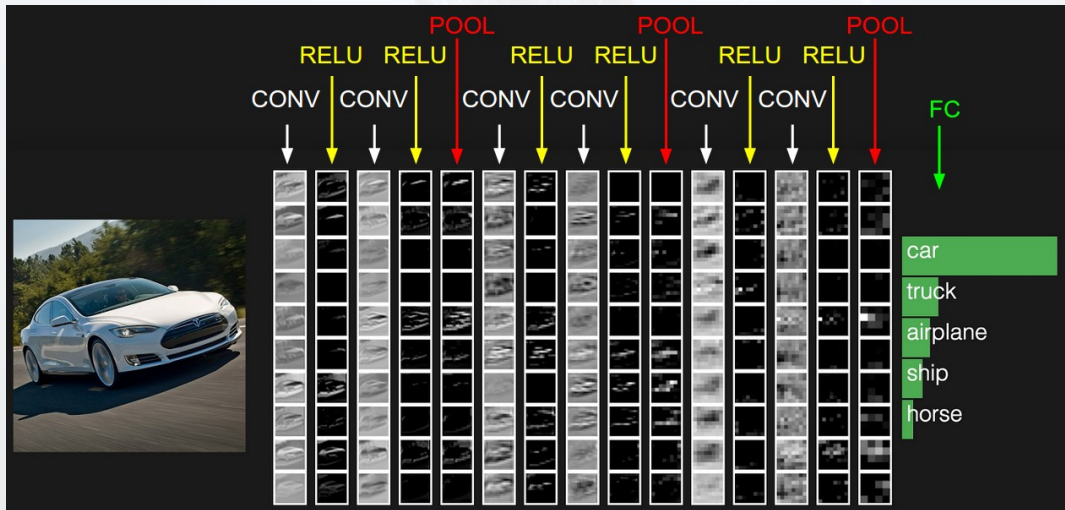




- Input volume: **32x32x3**
- 10 **5x5** filters with stride 1, pad 2
- **Output volume size: ?**

- Input volume: **32x32x3**
- 10 **5x5** filters with stride 1, pad 2
- **Number of parameters in this layer?**

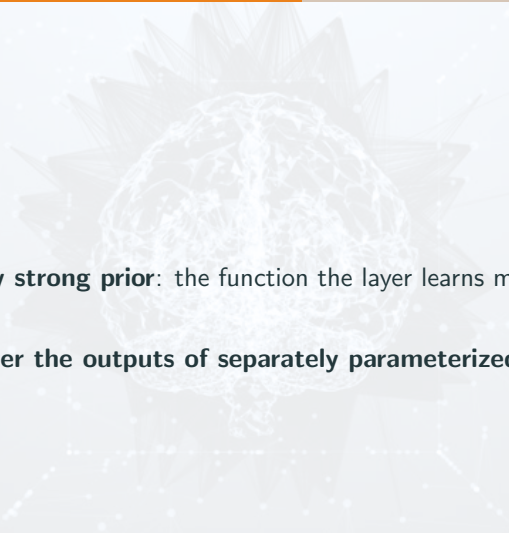
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K
 - Their spatial extent F
 - The stride S
 - The amount of zero padding P
- Produces a volume of size $W_2 \times H_2 \times D_2$
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

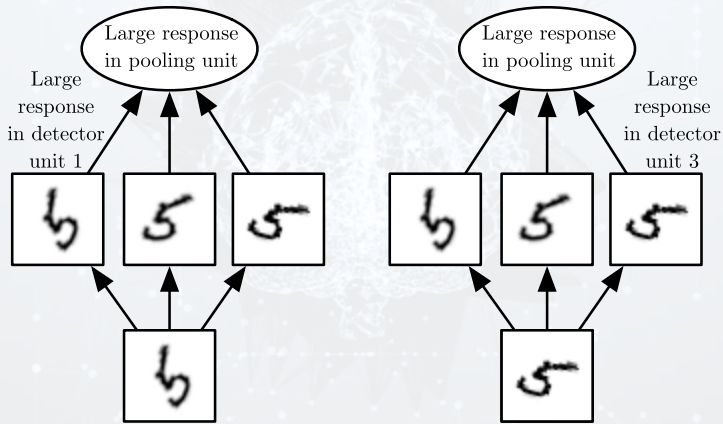


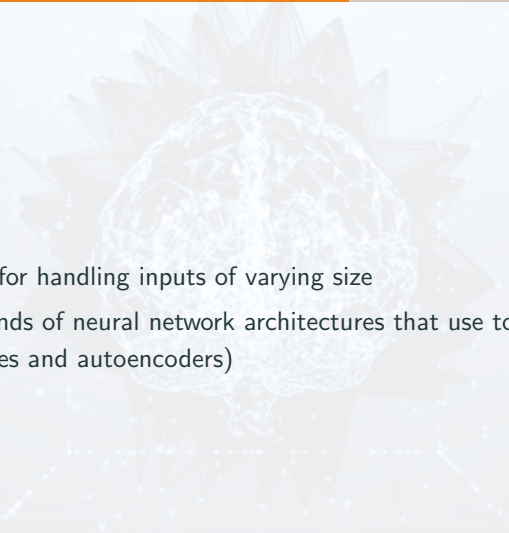
Pooling

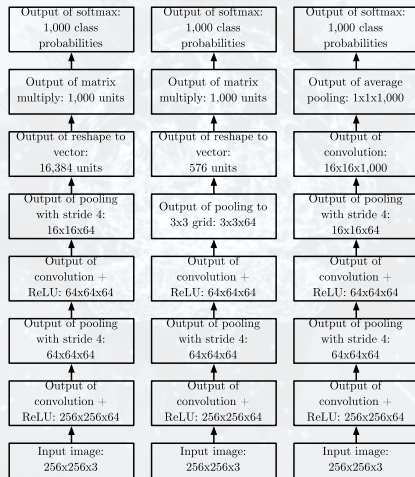


- **Pooling function:** replaces the output of the net at a certain location with a summary statistic of the nearby outputs
- The representation becomes approximately invariant to small translations of the input
- We care whether some feature is present rather than where it is

- 
- A large, faint, light-colored lotus flower is centered in the background of the slide. The lotus has many layers of petals, and its center is a complex, web-like structure. The background is a light gray with a subtle pattern of small white dots and lines, resembling a neural network or a starry sky.
- Adding an **infinitely strong prior**: the function the layer learns must be **invariant to small translations**
 - What if we **pool over the outputs of separately parameterized convolutions**?

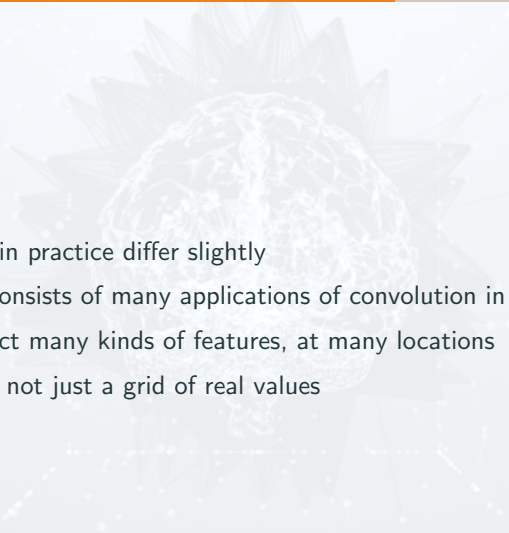


- 
- A faint, stylized background image of a human brain with a network of white lines representing neural connections or data flow, set against a light gray background.
- Pooling is essential for handling inputs of varying size
 - Complicate some kinds of neural network architectures that use top-down information (Boltzmann machines and autoencoders)

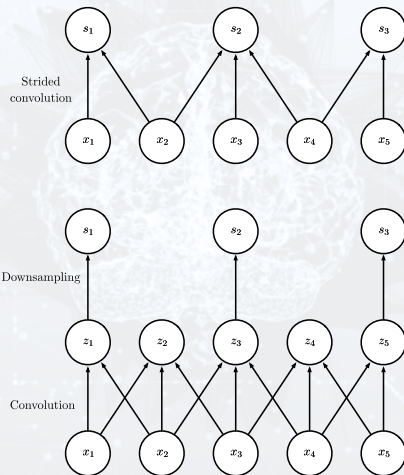


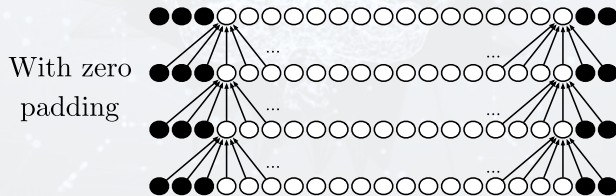
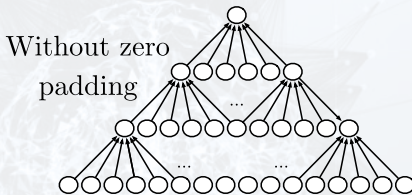


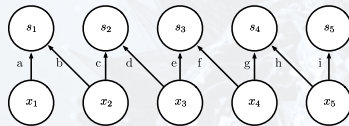
Variants of the Basic Convolution Function

- 
- A faint, stylized background graphic of a lotus flower, rendered in a light gray or white color, centered behind the text.
- The functions used in practice differ slightly
 - An operation that consists of many applications of convolution in parallel
 - At each layer: extract many kinds of features, at many locations
 - The input is usually not just a grid of real values

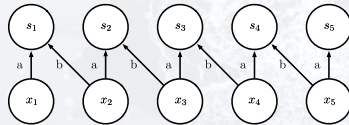
- **Stride**
- **Padding** or no padding?
 - **Valid** convolution
 - **Same** convolution
 - **Full** convolution
- **Unshared** convolutions



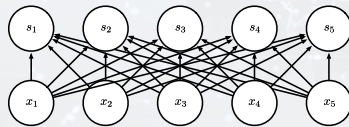




Local connection:
like convolution,
but no sharing

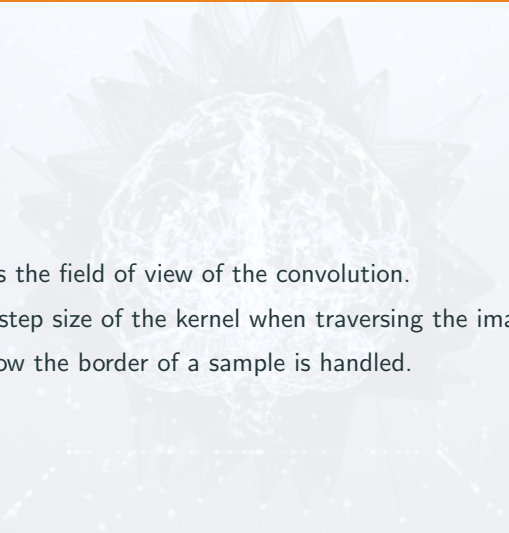


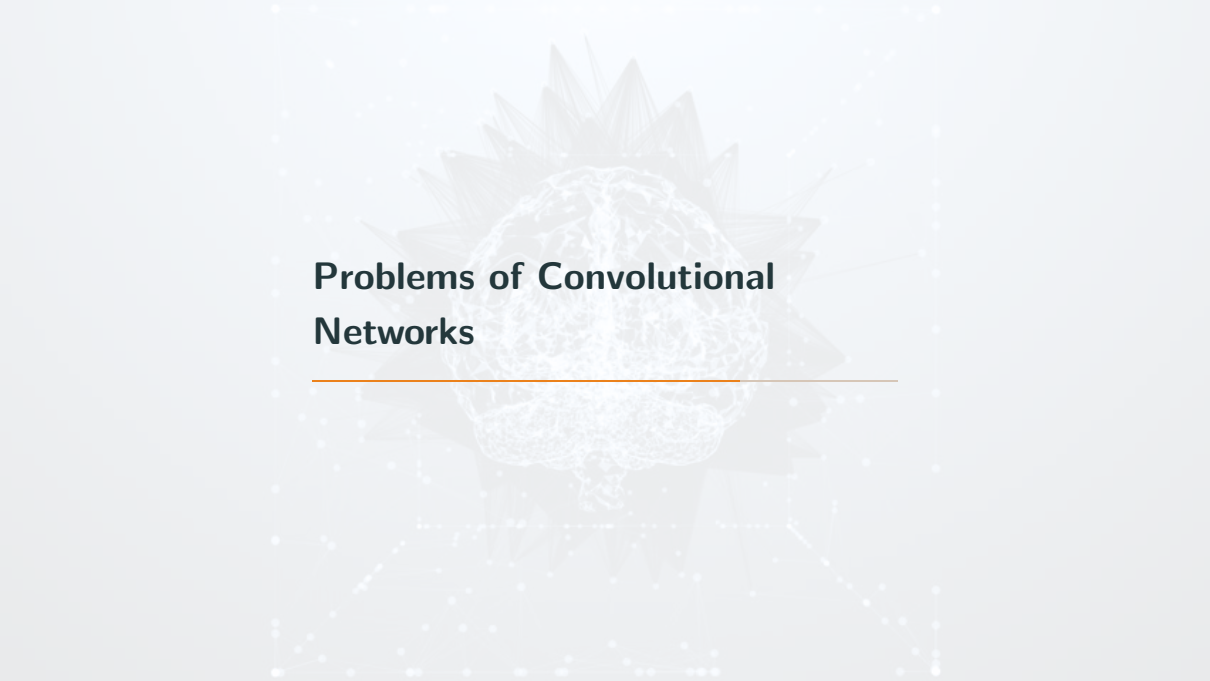
Convolution



Fully connected

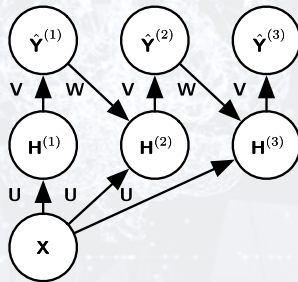
- Multiplication matrix is **sparse**
- Duplication in the matrix

- 
- A decorative background image featuring a stylized, light-colored lotus flower in the center. The lotus is surrounded by a network of white dots connected by thin lines, resembling a neural network or a data visualization. The entire background is a light gray color.
- **Kernel size:** defines the field of view of the convolution.
 - **Stride:** defines the step size of the kernel when traversing the image.
 - **Padding:** defines how the border of a sample is handled.



Problems of Convolutional Networks

- CNN can be used to output high-dimensional structured object



- 1D: audio waveform, skeleton animation data
- 2D: audio data (FFT), color image data
- 3D: Medical imaging data (CT scans), color video data



Random or Unsupervised Features

Training CNN

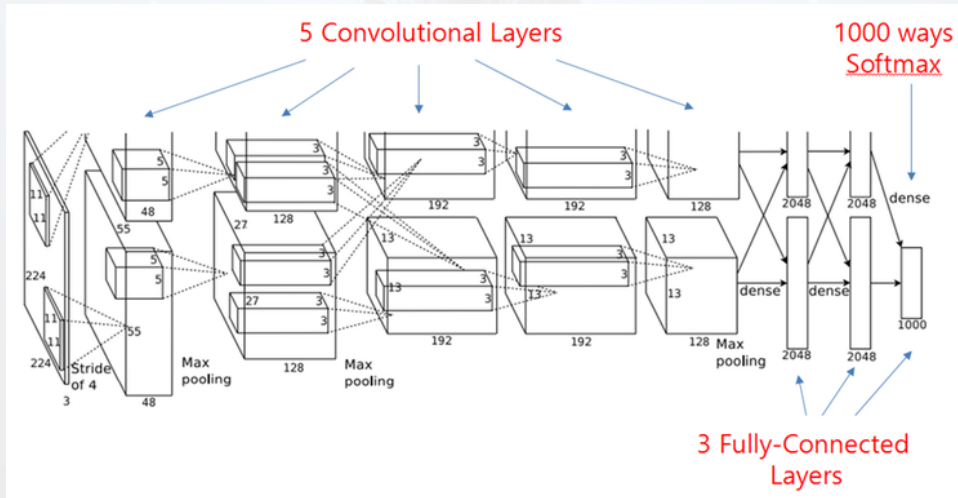
- Learning the features
- Unsupervised training
- Used features that are not trained in a supervised fashion

Basic strategies for obtaining convolution kernels

- Initialize them randomly
- Design by hand
- Learn the kernels with an unsupervised criterion

Case Study

Full (simplified) AlexNet architecture



227x227x3 INPUT

55x55x96 CONV1: 96 11x11 filters at stride 4, pad 0

27x27x96 MAX POOL1: 3x3 filters at stride 2

27x27x96 NORM1: Normalization layer

27x27x256 CONV2: 256 5x5 filters at stride 1, pad 2

13x13x256 MAX POOL2: 3x3 filters at stride 2

13x13x256 NORM2: Normalization layer

13x13x384 CONV3: 384 3x3 filters at stride 1, pad 1

13x13x384 CONV4: 384 3x3 filters at stride 1, pad 1

13x13x256 CONV5: 256 3x3 filters at stride 1, pad 1

6x6x256 MAX POOL3: 3x3 filters at stride 2

4096 FC6: 4096 neurons

4096 FC7: 4096 neurons

1000 FC8: 1000 neurons (class scores)

- First use of ReLU
- Used Norm layers (not common anymore) - heavy data augmentation
- Dropout 0.5
- Batch size 128
- SGD Momentum 0.9
- Learning rate $1e-2$, reduced by 10 manually when val accuracy plateaus
- L2 weight decay $5e-4$
- 7 CNN ensemble: 18.2% $\rightarrow$ 15.4%

Modified version of AlexNet

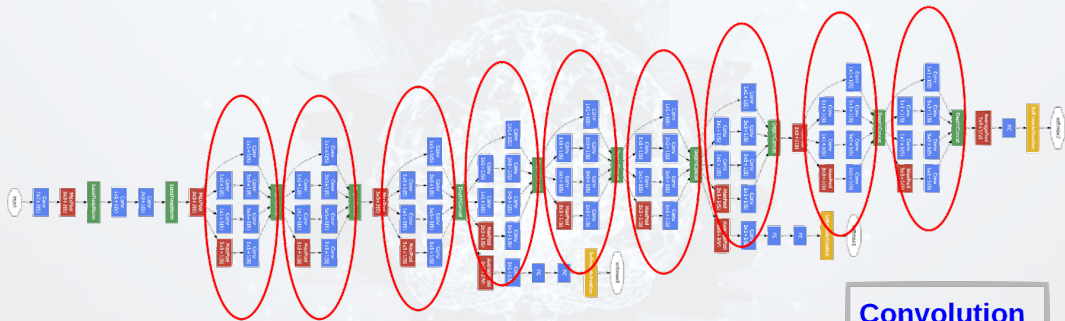
- **CONV1**: change from (11x11 stride 4) to (7x7 stride 2)
- **CONV3,4,5**: instead of 384, 384, 256 filters use 512, 1024, 512
- ImageNet top 5 error: **15.4%** - $\downarrow$ 14.8%

[Simonyan and Zisserman, 2014]

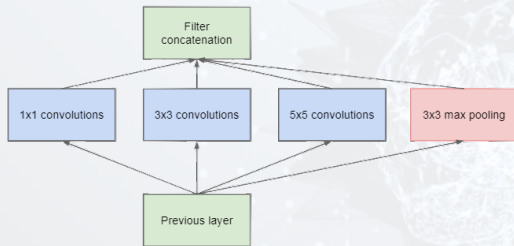
- Only 3x3 CONV stride 1, pad 1 and 2x2 MAX POOL stride 2
- **11.2% top 5 error in ILSVRC 2013 → 7.3% top 5 error**
- Number of parameters?

- INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0
- CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$
- CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$
- POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0
- CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$
- CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$
- POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
- POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
- POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0 (not counting biases)
- CONV3-512: [14x14x512] memory: $14*14*512=100K$
- CONV3-512: [14x14x512] memory: $14*14*512=100K$
- CONV3-512: [14x14x512] memory: $14*14*512=100K$
- POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0
- FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$
- FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$
- FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

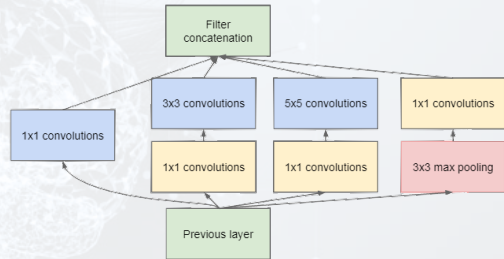
- TOTAL memory: $24M * 4 \text{ bytes} = 93MB$ / image (only forward! *2 for bwd)
- TOTAL params: **138M parameters**



Convolution
Pooling
Softmax
Concat/Normalize



(a) Inception module, naïve version



(b) Inception module with dimension reductions

ILSVRC 2014 winner (6.7% top 5 error)

- Fun features
 - Only 5 million params! (Removes FC layers completely)
- Compared to AlexNet
 - 12X less params
 - 2x more compute - 6.67% (vs. 16.4%)